

CS 578: CYBER-SECURITY

PART I – INTERNET ECOSYSTEM

Sanghyun Hong

sanghyun.hong@oregonstate.edu



Oregon State
University



TRUE AI
Trustworthy and Responsible AI

ANNOUNCEMENT

- Term-project and in-class presentation sign-up
 - Close on 9/30 (Wed)
 - On Google Sheet (link is available on the course website)
 - No team on the sheet = -2% of your term grade
 - No presentation sign-up / no slot left = 0 pts for the component (-15%)

RECAP

- The security mindset
 - Schneier: How does an adversary think? = How to break it?
 - Anderson: Who will be motivated to fix security problems? = How to set incentives?
- Make the Internet secure
 - Secure the communication between two parties (SSL/TLS)
 - Crypto algorithms are secure; but **their assumptions are broken**
 - Censys and Mining Your Ps and Qs

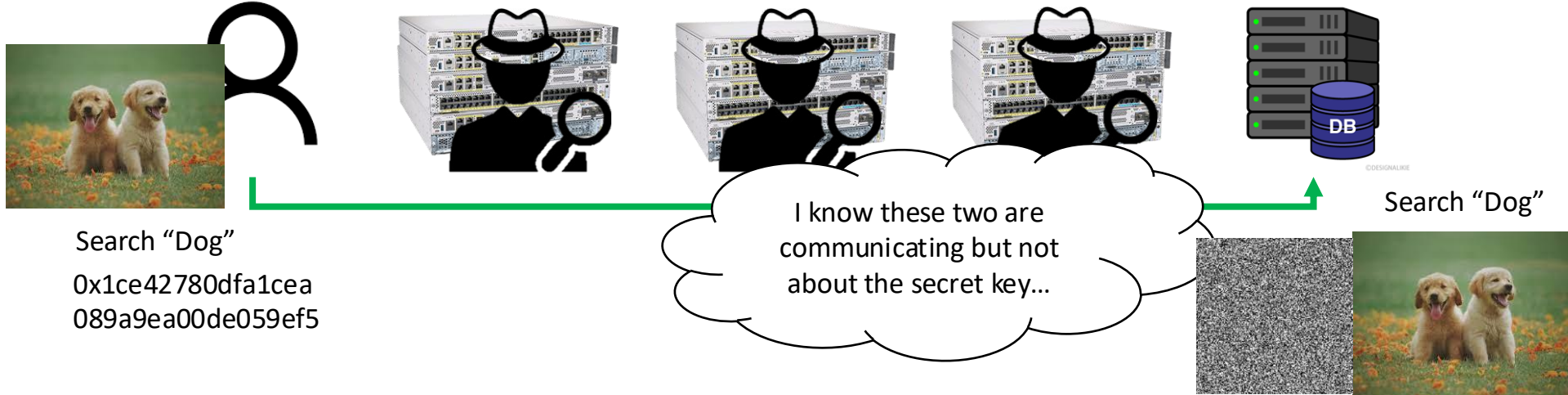
TWO MEASUREMENT STUDIES: ON SSL CERTIFICATES AND DNSSEC

RECAP ON SECURING THE INTERNET INFRASTRUCTURE

THE INTERNET WITH SECURE CONNECTIONS

Middle mans never know
DH exchange keys!!

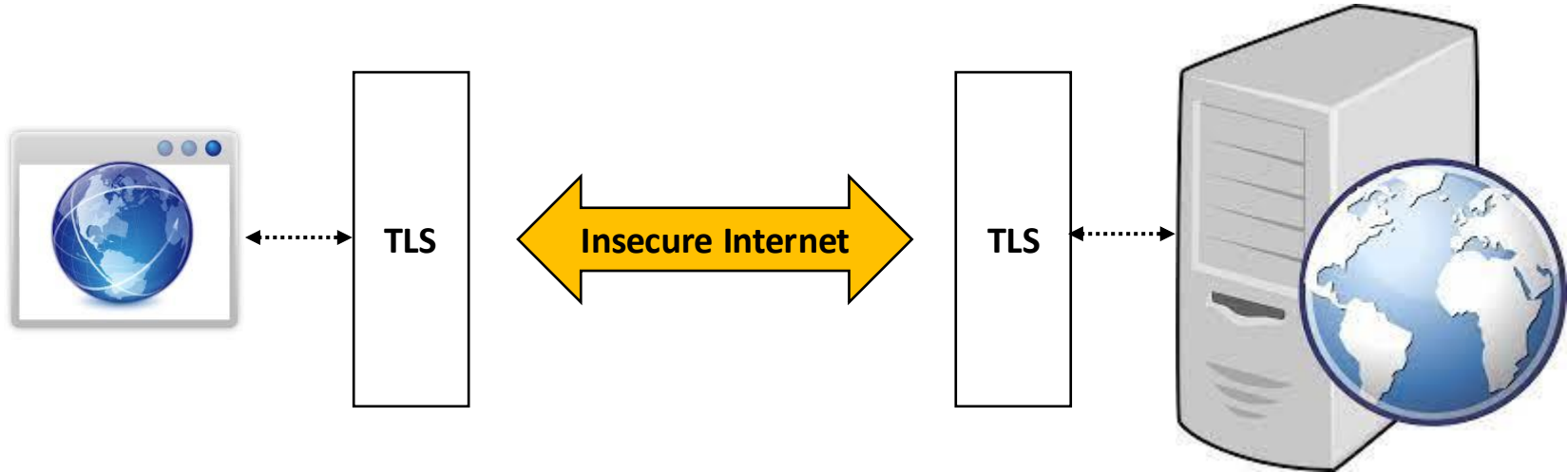
Check certificate, exchange keys, apply encryption with HMAC



**The Middlemen Will Only See the Encrypted Contents
They Will **Never** Know the Secret Key ...**

SSL/TLS ENABLES SECURE CONNECTIONS: HTTPS EXAMPLE

- Suppose we use HTTPs (instead of HTTP)



SSL/TLS ENABLES SECURE CONNECTIONS: HTTPS EXAMPLE



Run **TLS handshake** to establish a secure channel



©DESIGNAUXIE

PRIMER ON MAKING A SECURE CONNECTION = TLS HANDSHAKE

TLS HANDSHAKE

- Procedure overview

1. [Client] Sends “hello” to the server
2. [Server] replies “hello” to the client
 - Server sends its certificate and public key to the client
 - Server performs the “key exchange” procedure
3. [Client] performs the “key exchange” procedure
4. [Client] decides the cipher spec to use for this communication
5. [Client] sends the encrypted handshake message to the server
6. [Server] sets its cipher spec to the one received from the client
7. [Server] sends the encrypted handshake message to the client
8. Done, now both can communicate over encrypted data

TLS HANDSHAKE

- Procedure overview

1. [Client] Sends “hello” to the server
2. [Server] replies “hello” to the client
 - Server sends its certificate and public key to the client
 - Server performs the “key exchange” procedure
3. [Client] performs the “key exchange” procedure
4. [Client] decides the cipher spec to use for this communication
5. [Client] sends the encrypted handshake message to the server
6. [Server] sets its cipher spec to the one received from the client
7. [Server] sends the encrypted handshake message to the client
8. Done, now both can communicate over encrypted data

HANDSHAKE STEP I – CLIENT HELLO

- The first message a client sends to the server
 - It sends an SSL/TLS version, a random number, an available cipher suite, ...

```
00000000 16 03 01 01 44 01 00 01 40 03 03 95 8b 02 ec f4 ....D.
00000010 ca 4d 7d 98 6b 9e 3f 45 8b fa 92 10 f0 9c 2c aa .M}.k.
00000020 bf 27 f0 52 b0 97 6c f0 6c a2 a9 20 bc b7 86 80 .'R..
00000030 f2 f1 71 9f e0 7e 7e 4c c2 51 88 e7 72 2d e0 3c ..q..~
00000040 ca cc fa 2c 99 dc b9 56 d0 80 bd 91 00 62 13 02 ....,
00000050 13 03 13 01 c0 30 c0 2c c0 28 c0 24 c0 14 c0 0a .....0
00000060 00 9f 00 6b 00 39 cc a9 cc a8 cc aa ff 85 00 c4 ...k.9
00000070 00 88 00 81 00 9d 00 3d 00 35 00 c0 00 84 c0 2f .....
00000080 c0 2b c0 27 c0 23 c0 13 c0 09 00 9e 00 67 00 33 .+.'.#
00000090 00 be 00 45 00 9c 00 3c 00 2f 00 ba 00 41 c0 11 ...E..
000000A0 c0 07 00 05 00 04 c0 12 c0 08 00 16 00 0a 00 ff .....
000000B0 01 00 00 95 00 2b 00 09 08 03 04 03 03 02 03 .....+
000000C0 01 00 33 00 26 00 24 00 1d 00 20 ba 53 26 b5 f2 ..3.&.
000000D0 19 5d b0 e0 b5 f4 30 c0 73 e9 2a 1d 86 72 d5 29 .].....
000000E0 6e fc 32 3f d3 0f 31 d6 e2 57 61 00 00 18 00 n.2?..
000000F0 16 00 00 13 77 77 77 2e 6f 72 65 67 6f 6e 73 74 ....ww
00000100 61 74 65 2e 65 64 75 00 0b 00 02 01 00 00 0a 00 ate.ed
00000110 0a 00 08 00 1d 00 17 00 18 00 19 00 0d 00 18 00 .....
00000120 16 08 06 06 01 06 03 08 05 05 01 05 03 08 04 04 .....
00000130 01 04 03 02 01 02 03 00 10 00 0e 00 0c 02 68 32 .....
00000140 08 68 74 74 70 2f 31 2e 31 .http/

✓ TLSv1.2 Record Layer: Handshake Protocol: Client Hello
  Content Type: Handshake (22)
  Version: TLS 1.0 (0x0301)
  Length: 324
  ✓ Handshake Protocol: Client Hello
    Handshake Type: Client Hello (1)
    Length: 320
    Version: TLS 1.2 (0x0303)
    > Random: 958b02ecf4ca4d7d986b9e3f458bfa9210f09c2caabf27f052b0976cf06ca2a9
    Session ID Length: 32
    Session ID: bcb78680f2f1719fe07e7e4cc25188e7722de03ccaccfa2c99dcb956d080bd91
    Cipher Suites Length: 98
    > Cipher Suites (49 suites)
    Compression Methods Length: 1
    > Compression Methods (1 method)
    Extensions Length: 149
  ✓ Extension: supported_versions (len=9)
    Type: supported_versions (43)
    Length: 9
    Supported Versions length: 8
    Supported Version: TLS 1.3 (0x0304)
    Supported Version: TLS 1.2 (0x0303)
    Supported Version: TLS 1.1 (0x0302)
    Supported Version: TLS 1.0 (0x0301)
```

HANDSHAKE STEP I – CLIENT HELLO

- It sends supported cipher suites:
 - TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256
ECDHE_RSA_AES_128_GCM_SHA256

```
Cipher Suites (49 suites)
Number
Cipher Suite: TLS_AES_256_GCM_SHA384 (0x1302)
Cipher Suite: TLS_CHACHA20_POLY1305_SHA256 (0x1303)
Cipher Suite: TLS_AES_128_GCM_SHA256 (0x1301)
Cipher Suite: TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 (0xc030)
Cipher Suite: TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384 (0xc02c)
Cipher Suite: TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384 (0xc028)
Cipher Suite: TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA384 (0xc024)
Cipher Suite: TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA (0xc014)
Cipher Suite: TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA (0xc00a)
Cipher Suite: TLS_DHE_RSA_WITH_AES_256_GCM_SHA384 (0x009f)
Cipher Suite: TLS_DHE_RSA_WITH_AES_256_CBC_SHA256 (0x006b)
Cipher Suite: TLS_DHE_RSA_WITH_AES_256_CBC_SHA (0x0039)
Cipher Suite: TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256 (0xc0a9)
Cipher Suite: TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256 (0xc0a8)
Cipher Suite: TLS_DHE_RSA_WITH_CHACHA20_POLY1305_SHA256 (0xc0aa)
Cipher Suite: Unknown (0xff85)
Cipher Suite: TLS_DHE_RSA_WITH_CAMELLIA_256_CBC_SHA256 (0x00c4)
Cipher Suite: TLS_DHE_RSA_WITH_CAMELLIA_256_CBC_SHA (0x0088)
Cipher Suite: TLS_GOSTR341001_WITH_28147_CNT_IMIT (0x0081)
Cipher Suite: TLS_RSA_WITH_AES_256_GCM_SHA384 (0x009d)
Cipher Suite: TLS_RSA_WITH_AES_256_CBC_SHA256 (0x003d)
Cipher Suite: TLS_RSA_WITH_AES_256_CBC_SHA (0x0035)
Cipher Suite: TLS_RSA_WITH_CAMELLIA_256_CBC_SHA256 (0x00c0)
Cipher Suite: TLS_RSA_WITH_CAMELLIA_256_CBC_SHA (0x0084)
Cipher Suite: TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 (0xc02f)
Cipher Suite: TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 (0xc02b)
Cipher Suite: TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256 (0xc027)
Cipher Suite: TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA256 (0xc023)
Cipher Suite: TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA (0xc013)
Cipher Suite: TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA (0xc009)
Cipher Suite: TLS_DHE_RSA_WITH_AES_128_GCM_SHA256 (0x009e)
Cipher Suite: TLS_DHE_RSA_WITH_AES_128_CBC_SHA256 (0x0067)
Cipher Suite: TLS_DHE_RSA_WITH_AES_128_CBC_SHA (0x0033)
Cipher Suite: TLS_DHE_RSA_WITH_CAMELLIA_128_CBC_SHA256 (0x00be)
Cipher Suite: TLS_DHE_RSA_WITH_CAMELLIA_128_CBC_SHA (0x0045)
Cipher Suite: TLS_RSA_WITH_AES_128_GCM_SHA256 (0x009c)
Cipher Suite: TLS_RSA_WITH_AES_128_CBC_SHA256 (0x003c)
Cipher Suite: TLS_RSA_WITH_AES_128_CBC_SHA (0x002f)
Cipher Suite: TLS_RSA_WITH_CAMELLIA_128_CBC_SHA256 (0x00ba)
```

HANDSHAKE STEP II – SERVER HELLO

- The first message a client sends to the server
 - It sends an SSL/TLS version, a random number, an available cipher suite, ...

```
✓ TLSv1.2 Record Layer: Handshake Protocol: Server Hello
  Content Type: Handshake (22)
  Version: TLS 1.2 (0x0303)
  Length: 102
  ✓ Handshake Protocol: Server Hello
    Handshake Type: Server Hello (2)
    Length: 98
    Version: TLS 1.2 (0x0303)
    > Random: 7937be8da9875cf054f0ed18b7efec590e2fb8823ffb7afb87fdffed322822dc
    Session ID Length: 32
    Session ID: 6adeb8c9532bf74b3f5d9940e83f470e46ac3f49054c667dfe8255a6342bea6e
    Cipher Suite: TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 (0xc02f)
    Compression Method: null (0)
```

- The server choose a cipher based on the client's availability
 - **Chosen:** TLS_ECDHE_RSA_AES_128_GCM_SHA256

HANDSHAKE STEP III – SERVER CERTIFICATE

- The first message a client sends to the server
 - It sends an SSL/TLS version, a random number, an available cipher suite, ...
- The server choose a cipher based on the client's availability
 - **Chosen:** TLS_ECDHE_RSA_AES_128_GCM_SHA256
- The server next sends the certificate information to the client
 - It sends a full chain (PKI) of digital certificates

```

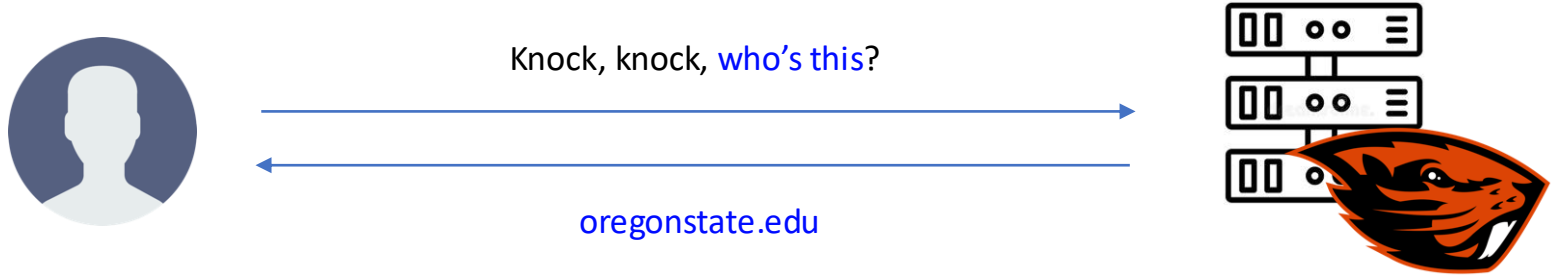
v TLSv1.2 Record Layer: Handshake Protocol: Certificate
  Content Type: Handshake (22)
  Version: TLS 1.2 (0x0303)
  Length: 6037
  v Handshake Protocol: Certificate
    Handshake Type: Certificate (11)
    Length: 6033
    Certificates Length: 6030
  v Certificates (6030 bytes)
    Certificate Length: 1994
    > Certificate: 308207c6308206aea003020102021030bc9131f05e7eef26b3844d426b816c300d06092a... (id-at-commonName=oregonstate.edu,id-at-organizationName
    Certificate Length: 1533
    > Certificate: 308205f9308203e1a00302010202104720d0fa85461a7e17a1640291846374300d06092a... (id-at-commonName=InCommon RSA Server CA,id-at-organizat
    Certificate Length: 1413
    > Certificate: 3082058130820469a00302010202103972443af922b751d7d36c10dd313595300d06092a... (id-at-commonName=USERTrust RSA Certification Authority,
    Certificate Length: 1078
    > Certificate: 308204323082031aa003020102020101300d06092a864886f70d0101050500307b310b30... (id-at-commonName=AAA Certificate Services,id-at-organiz

```

PRIMER ON DIGITAL CERTIFICATE AND PUBLIC KEY INFRASTRUCTURE

HOW CAN WE CHECK THE AUTHENTICITY?

- Can we check the other end is the one that we want to talk with?



We Need Some Ways to Check If They Are OSU (Authenticity)!

HOW CAN WE CHECK THE AUTHENTICITY?

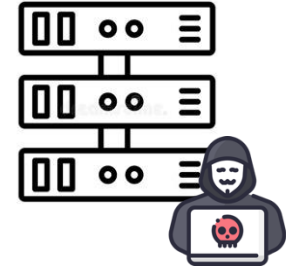
- Can we check the other end is the one that we want to talk with?



Knock, knock, *who's this?*



oregonstate.edu, just believe what I said!



We Need Some Ways to Check If They Are OSU (Authenticity)!

HOW DO WE DO THAT IN THE REAL-LIFE?

 **OREGON**  **USA**
DRIVER LICENSE

4d NO **A123456**

www.oregonstate.edu
0x83823787832a87b876
e67fe67e6da

4b EXP **12/12/2026** 15 SEX **M**
4A ISS **03/16/2018** 16 HGT **6'-02"**
10 FIRST **03/16/2018** 17 WGT **250 lb**
5 DD **ZA0000089** 18 EYES **BRO**

9 CLASS **C**
9a END **M**
12 REST **BD**

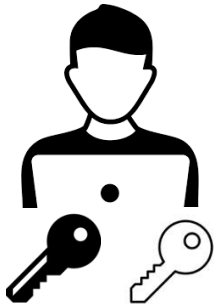
Signature 
3 DOB **12/12/1979** **VETERAN** 

HOW CAN WE DO THIS FOR ONLINE COMMUNICATION?

- Intuition
 - Need an identification mechanism
 - Need information that we can use to verify the sender
- Solution
 - Let's do this with RSA cryptography algorithm
 - Let “oregonstate.edu” publicize the public key
 - Let “oregonstate.edu” share their info. and signed by their private key
 - Then let clients
 - compute the signature over the shared info
 - compare it with the signature received from the server

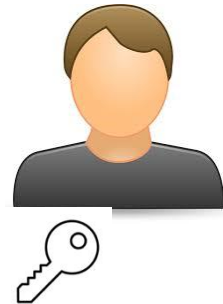
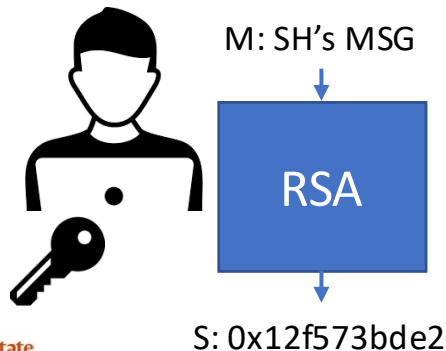
DIGITAL SIGNATURE AND RSA

- Digital signature
 - A mathematical scheme for verifying the **authenticity** of digital messages
 - RSA can be used for “**signing**”
- Encryption and decryption for “**signing**”
 - Encryption is applying the **private exponent** to a plaintext: $C = M^d \bmod N$
 - Decryption is applying the **public exponent** to a ciphertext: $M = C^e \bmod N$



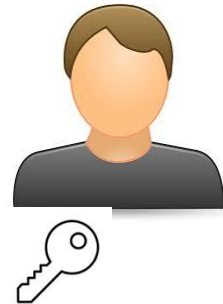
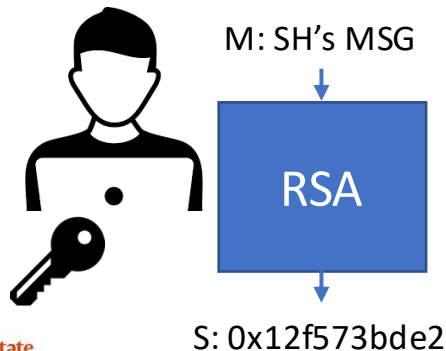
DIGITAL SIGNATURE AND RSA

- Digital signature
 - A mathematical scheme for verifying the **authenticity** of digital messages
 - RSA can be used for “**signing**”
- Encryption and decryption for “**signing**”
 - Encryption is applying the **private exponent** to a plaintext: $C = M^d \bmod N$
 - Decryption is applying the **public exponent** to a ciphertext: $M = C^e \bmod N$



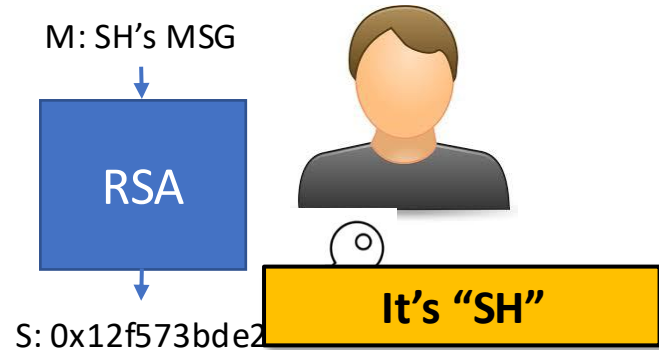
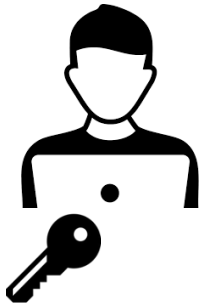
DIGITAL SIGNATURE AND RSA

- Digital signature
 - A mathematical scheme for verifying the **authenticity** of digital messages
 - RSA can be used for “**signing**”
- Encryption and decryption for “**signing**”
 - Encryption is applying the **private exponent** to a plaintext: $C = M^d \bmod N$
 - Decryption is applying the **public exponent** to a ciphertext: $M = C^e \bmod N$



DIGITAL SIGNATURE AND RSA

- Digital signature
 - A mathematical scheme for verifying the **authenticity** of digital messages
 - RSA can be used for “**signing**”
- Encryption and decryption for digital signature
 - Encryption is applying the **private exponent** to a plaintext: $C = M^d \bmod N$
 - Decryption is applying the **public exponent** to a ciphertext: $M = C^e \bmod N$



HOW CAN WE DO THIS FOR ONLINE COMMUNICATION?

- Intuition
 - Need an identification mechanism
 - Need information that we can use to verify the sender
- Solution: Public Key Infrastructure (PKI)
 - Let's do this with RSA cryptography algorithm
 - Let “oregonstate.edu” publicize the public key
 - Let “oregonstate.edu” share their info. and signed by their private key (= we create a digital certificate)
 - Then let clients
 - compute the signature over the shared info
 - compare it with the signature received from the server

THE INFO: DIGITAL CERTIFICATE

- A file that contains
 - Entity info (CN)
 - Issuer info (CN)
 - Public key
 - Signature

General

Details

Issued To

Common Name (CN)	oregonstate.edu
Organization (O)	Oregon State University
Organizational Unit (OU)	<Not Part Of Certificate>

Issued By

Common Name (CN)	InCommon RSA Server CA
Organization (O)	Internet2
Organizational Unit (OU)	InCommon

Validity Period

Issued On	Sunday, June 5, 2022 at 5:00:00 PM
Expires On	Tuesday, June 6, 2023 at 4:59:59 PM

Fingerprints

SHA-256 Fingerprint	7B 57 A4 91 B0 06 29 2E 8E 54 04 FB BB F6 F8 4F 09 56 15 C0 20 59 37 9F E9 F1 A4 27 DC B6 F4 E1
SHA-1 Fingerprint	FC EE 7C 4B AA 30 8F A6 03 E2 22 C5 31 FF 6C C6 92 FF C3 8E

HOW TO CREATE A DIGITAL CERTIFICATE?

- Requester prepares a certificate request
 - Entity information
 - Public key
 - Signature (proving that I have the public key)

Certificate

CN: oregonstate.edu

Will use for:

*.oregonstate.edu

Public Key: 0x112233445566778899aabbccddeeff....
(beaver's public key)

Signature: 0xaabbccddeeff00112233445566778899
(using beaver's private key)

HOW TO CREATE A DIGITAL CERTIFICATE?

- Requester prepares a certificate request
 - Entity information
 - Public key
 - Signature (proving that I have the public key)

Get SHA256 sum of this part

Certificate

CN: oregonstate.edu

Will use for:

*.oregonstate.edu

Public Key: 0x112233445566778899aabbccddeeff....
(beaver's public key)

Sign it with the private key

Signature: 0xaabbccddeeff00112233445566778899
(using beaver's private key)

HOW TO CREATE A DIGITAL CERTIFICATE?

- Requester prepares a certificate request
 - Entity information
 - Public key
- Issuer verifies the requester information, and digitally sign the cert
 - Verify the entity information
 - Get a SHA-256 fingerprint of the certificate
 - Sign the fingerprint (with issuer's private key)
`RSA_encrypt(private_key, SHA-256(certificate))`

HOW TO CREATE A DIGITAL CERTIFICATE?

- Issuer verifies the requester information, and digitally sign the cert
 - Verify the entity information
 - Get a SHA-256 fingerprint of the certificate
 - Sign the fingerprint (with issuer's private key)

```
RSA_encrypt(private_key, SHA-256(certificate))
```

Get SHA256 sum of this part

Certificate

CN: oregonstate.edu

Will use for:

*.oregonstate.edu

Public Key: 0x112233445566778899aabbccddeeff....
(beaver's public key)

Sign it with the private key

Signature: 0xaabbccddeeff00112233445566778899
(using beaver's private key)

HOW TO CREATE A DIGITAL CERTIFICATE?

- Requester prepares a certificate request
 - Entity information
 - Public key
- Issuer verifies the requester information, and digitally sign the cert
 - Verify the entity information
 - Get a SHA-256 fingerprint of the certificate
 - Sign the fingerprint (with issuer's private key)
`RSA_encrypt(private_key, SHA-256(certificate))`
- Anyone with the public key can verify the result
 - Get issuer's public key from their certificate

CERTIFICATION CREATION DETAILS: STEP 1

- The certificate requesting entity fills
 - Entity information
 - Public Key



- Entity:
 - For google, its *.google.com
 - Can be your website address
- *.secure-ai.systems
 - also has a certificate

CN = oregonstate.edu

Certificate
CN: oregonstate.edu
Will use for:
 *.oregonstate.edu

Public Key: 0x112233445566778899aabbccddeeff....
 (beaver's public key)

Signature: 0xaabbccddeeff00112233445566778899
 (with beaver's private key)

CERTIFICATION CREATION DETAILS: STEP 2

- The issuer receives the certificate request and verifies:

- Entity

- Their identification
 - Owning the target domain name
 - Owning the public key



- The signature

- Decrypt the signature with public key
 - It must be the same as SHA256 sum
 - It proves their holding the private key

CN = oregonstate.edu

Certificate

CN: oregonstate.edu

Will use for:

*.oregonstate.edu

Public Key: 0x112233445566778899aabbccddeeff....
(beaver's public key)

Signature: 0xaabbccddeeff00112233445566778899
(with beaver's private key)

CERTIFICATION CREATION DETAILS: STEP 2

- The issuer receives the certificate request and verifies:

- Entity:

- Their identification
 - Owning the target domain name
 - etc...



InCommon®

- Then, fill issuer information

- Issuer information
 - Issuer public key

CN = oregonstate.edu

Certificate

CN: oregonstate.edu

Will use for:

*.oregonstate.edu

Public Key: 0x112233445566778899aabbccddeeff....
(beaver's public key)

Issuer: InCommon RSA

Public Key: 0x22334455667788990011aabbccddeeff

CERTIFICATION CREATION DETAILS: STEP 2

- The issuer receives the certificate request and verifies:

- Entity:

- Their identification
- Owning the target domain name
- etc...



InCommon®

- Then, fill issuer information

- Issuer information
- Issuer public key

- and then, sign the certificate

- Get SHA-256 of the certificate
- Attach it as a signature!

CN = oregonstate.edu

Certificate

CN: oregonstate.edu

Will use for:

*.oregonstate.edu

Public Key: 0x112233445566778899aabbccddeeff....
(beaver's public key)

Issuer: InCommon RSA

Public Key: 0x22334455667788990011aabbccddeeff

Signature: 0xffeeddccbbaa00112233445566778899
(InCommon RSA's private key)

THE CERTIFICATE ISSUED

- Now InCommon RSA verified
 - oregonstate.edu is owned by
 - Oregon State University
 - With a specific Public Key

▼ Subject Public Key Info

Subject Public Key Algorithm

Subject's Public Key

Field Value

Modulus (2048 bits):

C8 7D 2D A8 EB 12 59 6B 90 6D 4F 71 1E 4C FA C2
 F7 A1 EC F6 E6 0E 39 52 FF 69 C0 36 CD A9 74 6E
 60 72 C8 34 AF CC F7 6F 8E 66 D0 C5 0D E9 9C 66
 F0 B2 D1 D8 75 A7 B9 82 E5 E8 C3 3F 13 35 1E 1E
 71 F1 92 B4 40 07 EA 27 BE F9 9B AF E8 D2 E3 71
 E7 9C E7 2E 7A 6E 7E 5E 8E 7A 00 72 B6 2D 2B 8A

TRU-ALLab: CS 578 - Cybersecurity

General

Details

Issued To

Common Name (CN)	oregonstate.edu
Organization (O)	Oregon State University
Organizational Unit (OU)	<Not Part Of Certificate>

Issued By

Common Name (CN)	InCommon RSA Server CA
Organization (O)	Internet2
Organizational Unit (OU)	InCommon

Validity Period

Issued On	Sunday, June 5, 2022 at 5:00:00 PM
Expires On	Tuesday, June 6, 2023 at 4:59:59 PM

Fingerprints

SHA-256 Fingerprint	7B 57 A4 91 B0 06 29 2E 8E 54 04 FB BB F6 F8 4F 09 56 15 C0 20 59 37 9F E9 F1 A4 27 DC B6 F4 E1
SHA-1 Fingerprint	FC EE 7C 4B AA 30 8F A6 03 E2 22 C5 31 FF 6C C6 92 FF C3 8E

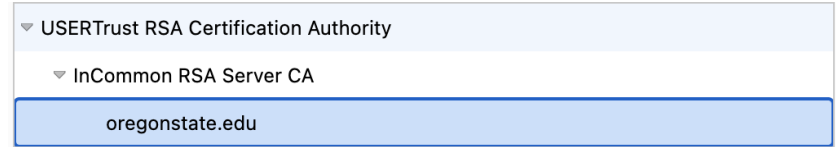


RECAP: OSU CERTIFICATE

- OSU owns “oregonstate.edu”
 - Verified by InCommon RSA
- Verification of the certificate
 - Use InCommon RSA’s public key
 - Where is it? It is written in InCommon RSA’s certificate
- But InCommon RSA, who will verify their identity?
 - InCommon RSA verifies “oregonstate.edu”
 - Who will verify InCommon RSA?

LET'S SEE IT FROM THE BROWSER

- “oregonstate.edu”
 - Verified by InCommon RSA Server CA
- InCommon RSA Server CA
 - Verified by USERTrust RSA Certificate Authority
- USERTrust RSA CA
 - Verified by **self**



TRUST CHAIN

- “oregonstate.edu”
 - Verified by InCommon RSA Server CA
- InCommon RSA Server CA
 - Verified by USERTrust RSA Certificate Authority
- USERTrust RSA CA
 - Verified by **self**

oregonstate.edu		InCommon RSA Server CA	
Subject Name			
Country	US		
State/Province	Oregon		
Organization	Oregon State University		
Common Name	oregonstate.edu		
Issuer Name			
Country	US		
State/Province	MI		
Locality	Ann Arbor		
Organization	Internet2		
Organizational Unit	InCommon		
Common Name	InCommon RSA Server CA		

TRUST CHAIN – CONT'D

- “oregonstate.edu”
 - Verified by InCommon RSA Server CA
- InCommon RSA Server CA
 - Verified by USERTrust RSA Certificate Authority
- USERTrust RSA CA
 - Verified by **self**

oregonstate.edu	InCommon RSA Server CA	USERTrust RSA Certification Authority
Subject Name		
Country	US	
State/Province	MI	
Locality	Ann Arbor	
Organization	Internet2	
Organizational Unit	InCommon	
Common Name	InCommon RSA Server CA	
Issuer Name		
Country	US	
State/Province	New Jersey	
Locality	Jersey City	
Organization	The USERTRUST Network	
Common Name	USERTrust RSA Certification Authority	

TRUST CHAIN – CONT'D

- “oregonstate.edu”
 - Verified by InCommon RSA Server CA
- InCommon RSA Server CA
 - Verified by USERTrust RSA Certification Authority
- USERTrust RSA CA
 - Verified by **self**

oregonstate.edu	InCommon RSA Server CA	USERTrust RSA Certification Authority
Subject Name		
Country	US	
State/Province	New Jersey	
Locality	Jersey City	
Organization	The USERTRUST Network	
Common Name	USERTrust RSA Certification Authority	
Issuer Name		
Country	US	
State/Province	New Jersey	
Locality	Jersey City	
Organization	The USERTRUST Network	
Common Name	USERTrust RSA Certification Authority	

TRUST CHAIN IN REAL-LIFE

- An example:
 - Student
 - Oregon resident
 - U.S. Citizen
- When issuing the student ID
 - We verify your Oregon ID...

TRUST CHAIN IN REAL-LIFE

- An example:
 - Student
 - Oregon resident
 - U.S. Citizen
- When issuing the student ID
 - Verify your Oregon ID...
- When issuing the Oregon Driver's License
 - Require either one of your birth certificate, previous Driver's License, or U.S. passport

TRUST CHAIN IN REAL-LIFE

- An example:
 - Student
 - Oregon resident
 - U.S. Citizen
- When issuing the student ID
 - Verify your Oregon ID...
- When issuing the Oregon Driver's License
 - Require either one of your birth certificate, previous Driver's License, or U.S. passport
- When issuing the U.S. passport
 - Require your birth certificate or previously issued passport..

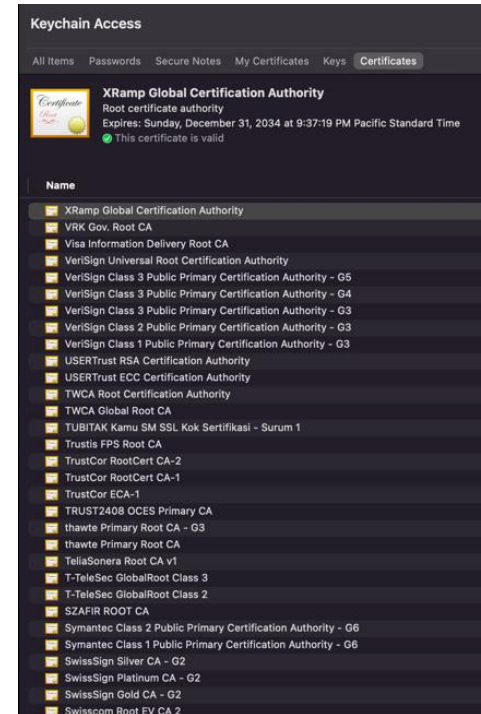
TRUST CHAIN IN REAL-LIFE

- An example:
 - Student
 - Oregon resident
 - U.S. Citizen
- When issuing the student ID
 - Verify your Oregon ID...
- When issuing the Oregon Driver's License
 - Require either one of your birth certificate, previous Driver's License, or U.S. passport
- When issuing the U.S. passport
 - Require your birth certificate or previously issued passport..

We need **someone** to **verify** the **originality** of the proving document...

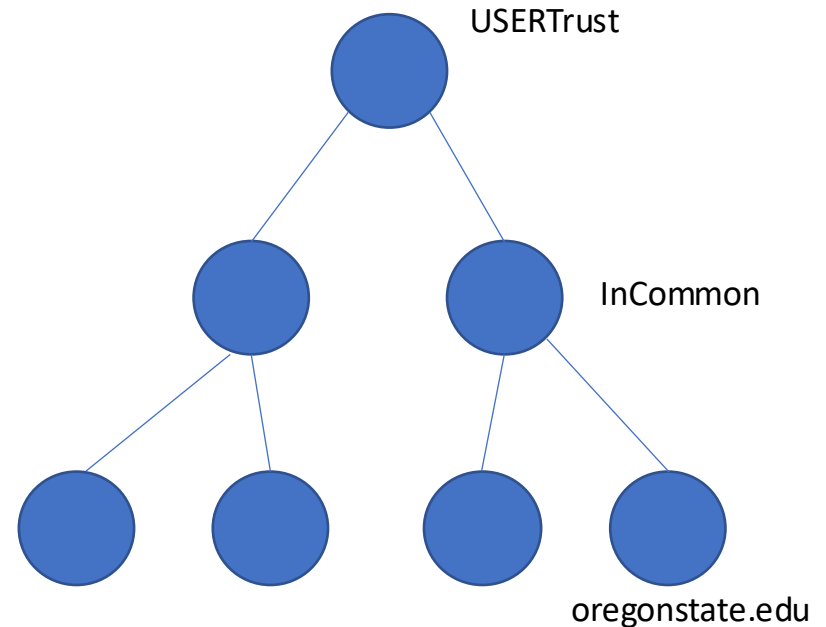
ROOT CERTIFICATE AUTHORITY (Root CA $\approx$ US IN PREV. EXAMPLE)

- Define small set of trustworthy certificate authorities
 - Private companies are authorized by some jurisdiction to run the CA company
 - Google Trust Service (GTS CA)
 - DigiCert
 - Verisign
 - etc..
- Trust their self-signed certificate
 - Stored in almost every computer machines



PUBLIC KEY INFRASTRUCTURE (PKI)

- An Infrastructure that provides public key with certificate chain
- Trust anchor: Root CA
 - Set a small set of entities use self-signed cert
- Verify the certificate chain!
 - Must verify the entire chain

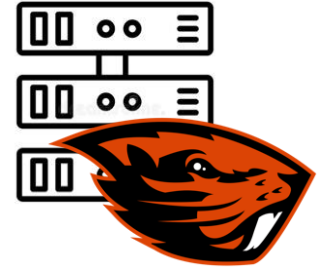


LET'S VERIFY OREGONSTATE.EDU

- Using the digital certificate!



Hey, are you oregonstate.edu?
Give me your certificate



LET'S VERIFY OREGONSTATE.EDU

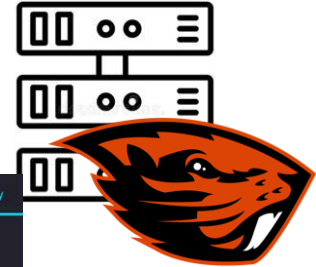
- Using the digital certificate!



Hey, are you oregonstate.edu?
Give me your certificate



Yes, I am oregonstate.edu!
Here's my cert (certificate)



oregonstate.edu		InCommon RSA Server CA
Subject Name		
Country	US	
State/Province	Oregon	
Organization	Oregon State University	
Common Name	oregonstate.edu	
Issuer Name		
Country	US	
State/Province	MI	
Locality	Ann Arbor	
Organization	Internet2	
Organizational Unit	InCommon	
Common Name	InCommon RSA Server CA	

oregonstate.edu		InCommon RSA Server CA	US
Subject Name			
Country	US		
State/Province	MI		
Locality	Ann Arbor		
Organization	Internet2		
Organizational Unit	InCommon		
Common Name	InCommon RSA Server CA		
Issuer Name			
Country	US		
State/Province	New Jersey		
Locality	Jersey City		
Organization	The USERTRUST Network		
Common Name	USERTrust RSA Certification Authority		

state.edu		InCommon RSA Server CA	USERTrust RSA Certification Authority
Subject Name			
Country	US		
State/Province	New Jersey		
Locality	Jersey City		
Organization	The USERTRUST Network		
Common Name	USERTrust RSA Certification Authority		
Issuer Name			
Country	US		
State/Province	New Jersey		
Locality	Jersey City		
Organization	The USERTRUST Network		
Common Name	USERTrust RSA Certification Authority		

LET'S VERIFY OREGONSTATE.EDU

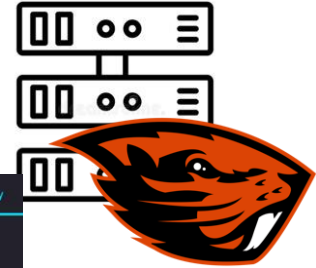
- Using the digital certificate!



Hey, are you oregonstate.edu?
Give me your certificate



Yes, I am oregonstate.edu!
Here's my cert



oregonstate.edu		InCommon RSA Server CA
Subject Name		
Country	US	
State/Province	Oregon	
Organization	Oregon State University	
Common Name	oregonstate.edu	
Issuer Name		
Country	US	
State/Province	MI	
Locality	Ann Arbor	
Organization	Internet2	
Organizational Unit	InCommon	
Common Name	InCommon RSA Server CA	

oregonstate.edu		InCommon RSA Server CA	US
Subject Name			
Country	US		
State/Province	MI		
Locality	Ann Arbor		
Organization	Internet2		
Organizational Unit	InCommon		
Common Name	InCommon RSA Server CA		
Issuer Name			
Country	US		
State/Province	New Jersey		
Locality	Jersey City		
Organization	The USERTRUST Network		
Common Name	USERTrust RSA Certification Authority		

state.edu		InCommon RSA Server CA	USERTrust RSA Certification Authority
Subject Name			
Country	US		
State/Province	New Jersey		
Locality	Jersey City		
Organization	The USERTRUST Network		
Common Name	USERTrust RSA Certification Authority		
Issuer Name			
Country	US		
State/Province	New Jersey		
Locality	Jersey City		
Organization	The USERTRUST Network		
Common Name	USERTrust RSA Certification Authority		

LET'S VERIFY OREGONSTATE.EDU

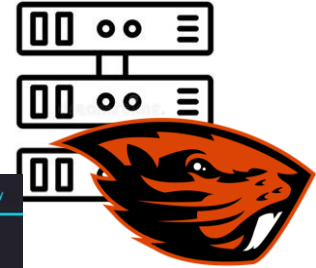
- Using the digital certificate!



Hey, are you oregonstate.edu?
Give me your certificate



Yes, I am oregonstate.edu!
Here's my cert



oregonstate.edu		InCommon RSA Server CA
Subject Name		
Country	US	
State/Province	Oregon	
Organization	Oregon State University	
Common Name	oregonstate.edu	
Issuer Name		
Country	US	
State/Province	MI	
Locality	Ann Arbor	
Organization	Internet2	
Organizational Unit	InCommon	
Common Name	InCommon RSA Server CA	

oregonstate.edu		InCommon RSA Server CA	US
Subject Name			
Country	US		
State/Province	MI		
Locality	Ann Arbor		
Organization	Internet2		
Organizational Unit	InCommon		
Common Name	InCommon RSA Server CA		
Issuer Name			
Country	US		
State/Province	New Jersey		
Locality	Jersey City		
Organization	The USERTRUST Network		
Common Name	USERTrust RSA Certification Authority		

state.edu		InCommon RSA Server CA	USERTrust RSA Certification Authority
Subject Name			
Country	US		
State/Province	New Jersey		
Locality	Jersey City		
Organization	The USERTRUST Network		
Common Name	USERTrust RSA Certification Authority		
Issuer Name			
Country	US		
State/Province	New Jersey		
Locality	Jersey City		
Organization	The USERTRUST Network		
Common Name	USERTrust RSA Certification Authority		

LET'S VERIFY OREGONSTATE.EDU

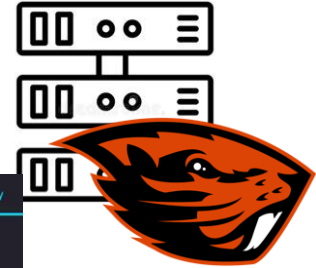
- Using the digital certificate!



Hey, are you oregonstate.edu?
Give me your certificate



Yes, I am oregonstate.edu!
Here's my cert



oregonstate.edu		InCommon RSA Server CA
Subject Name		
Country	US	
State/Province	Oregon	
Organization	Oregon State University	
Common Name	oregonstate.edu	
Issuer Name		
Country	US	
State/Province	MI	
Locality	Ann Arbor	
Organization	Internet2	
Organizational Unit	InCommon	
Common Name	InCommon RSA Server CA	

oregonstate.edu		InCommon RSA Server CA	USERTrust RSA Certification Authority
Subject Name			
Country	US		
State/Province	MI		
Locality	Ann Arbor		
Organization	Internet2		
Organizational Unit	InCommon		
Common Name	InCommon RSA Server CA		
Issuer Name			
Country	US		
State/Province	New Jersey		
Locality	Jersey City		
Organization	The USERTRUST Network		
Common Name	USERTrust RSA Certification Authority		

state.edu		InCommon RSA Server CA	USERTrust RSA Certification Authority
Subject Name			
Country	US		
State/Province	New Jersey		
Locality	Jersey City		
Organization	The USERTRUST Network		
Common Name	USERTrust RSA Certification Authority		
Issuer Name			
Country	US		
State/Province	New Jersey		
Locality	Jersey City		
Organization	The USERTRUST Network		
Common Name	USERTrust RSA Certification Authority		

PRIMER ON THE KEY EXCHANGE PROCEDURE

TLS HANDSHAKE

- Procedure overview

1. [Client] Sends “hello” to the server
2. [Server] replies “hello” to the client
 - Server sends its certificate and public key to the client
 - Server performs the “key exchange” procedure
3. [Client] performs the “key exchange” procedure
4. [Client] decides the cipher spec to use for this communication
5. [Client] sends the encrypted handshake message to the server
6. [Server] sets its cipher spec to the one received from the client
7. [Server] sends the encrypted handshake message to the client
8. Done, now both can communicate over encrypted data

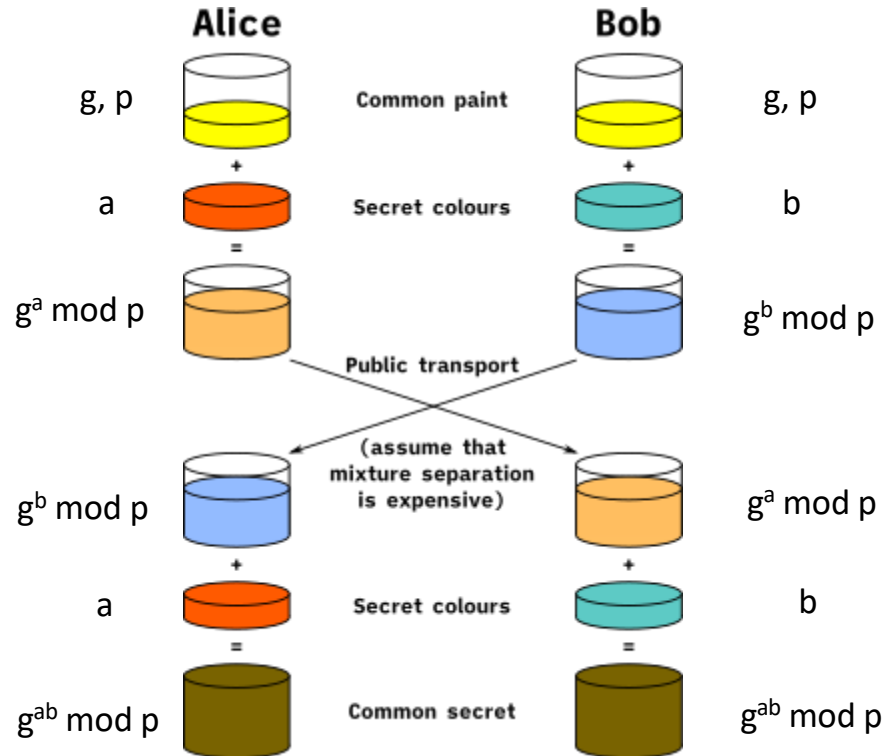
DIFFIE-HELLMAN KEY EXCHANGE

- Diffie-Hellman
 - A method of securely exchanging cryptographic keys over a public channel
 - Two parties can establish a shared secret (private) key over an insecure channel

DIFFIE-HELLMAN KEY EXCHANGE

- User A & User B agrees on g and p , where g and p are primes
- User A secretly chooses a , send $A = g^a \bmod p$
- User B secretly chooses b , send $B = g^b \bmod p$
- User A receives B , compute $B^a = (g^b)^a \bmod p = g^{ab} \bmod p$
- User B receives A , compute $A^b = (g^a)^b \bmod p = g^{ab} \bmod p$
- $g^{ab} \bmod p$ is our secret

DIFFIE-HELLMAN KEY EXCHANGE – CONT'D



DIFFIE-HELLMAN KEY EXCHANGE – CONT'D

- Security:
 - Based on the difficulty of mathematical problem of [discrete logarithm](#)
 - Example:
 - Given g, a, b, A, B , where
 - $g^a \bmod p = A$
 - $g^b \bmod p = B$
 - Can you compute a and b

DIFFIE-HELLMAN KEY EXCHANGE

- $g^{ab} \bmod p$ is our secret
- Suppose:
 - Attacker knows g , p , $A = g^a \bmod p$ and $B = g^b \bmod p$
 - $A+B = (g^a + g^b) \bmod p$
 - $AB = g^{(a+b)} \bmod p$
- Security:
 - Hard to compute g^{ab} from those values
 - Discrete logarithm; can you guess a from $A = g^a \bmod p$

DIFFIE-HELLMAN KEY EXCHANGE EXAMPLE

- $g = 5, p = 23$
- A chooses $a = 4$
 - $A = 5^4 \bmod 23 = 625 \bmod 23 = 4$
- B chooses $b = 3$
 - $B = 5^3 \bmod 23 = 125 \bmod 23 = 10$
- $B^4 = 10^4 \bmod 23 = 10000 \bmod 23 = 18$
- $A^3 = 4^3 \bmod 23 = 64 \bmod 23 = 18$
- $5^{(4*3)} = 5^{12} \bmod 23 = 18$

DIFFIE-HELLMAN KEY EXCHANGE: IMPLICATIONS

- Users are agreeing on two prime numbers
 - g, p
- User A chooses any integer a , nobody knows it
- User B chooses any integer b , nobody knows it
- By sharing $g^a \bmod P$ and $g^b \bmod p$
 - Both shares $g^{ab} \bmod P$ without leaking a nor b

**Two entities can interactively share a secret
without directly leaking the secrets to others**

TLS HANDSHAKE

- Procedure overview

1. [Client] Sends “hello” to the server
2. [Server] replies “hello” to the client
 - Server sends its certificate and public key to the client
 - Server performs the “key exchange” procedure
3. [Client] performs the “key exchange” procedure
4. [Client] decides the cipher spec to use for this communication
5. [Client] sends the encrypted handshake message to the server
6. [Server] sets its cipher spec to the one received from the client
7. [Server] sends the encrypted handshake message to the client
8. Done, now both can communicate over encrypted data

CERTIFICATE REVOCATION MEASUREMENT, IN THE WAKE OF HEARTBLEED

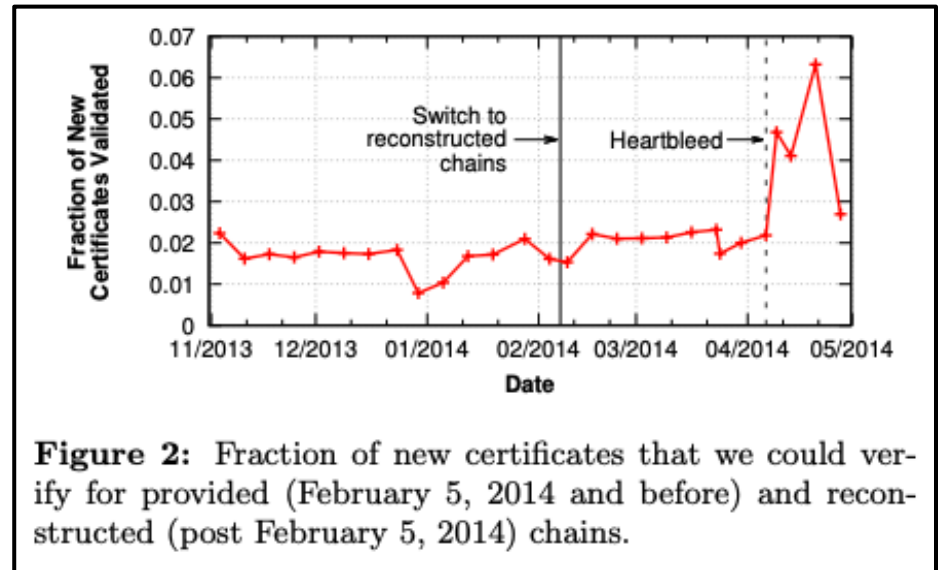
REVISIT: HANDSHAKE REQUIRES FORWARD SECURITY

- Forward Secrecy / Perfect Forward Secrecy
 - We want to keep all the communication secure
 - Even if the server's private key (i.e., the long-term key) has been breached
- Example of such breaches
 - Heartbleed (<https://heartbleed.com/>): CVE-2014-0160



IMPACT OF HEARTBLEED ON CERTIFICATE REVOCATION

- Check the trust chain
 - A large increase in the fraction of newly-appearing certificates
 - Many certificates are re-issued in the wake of heartbleed

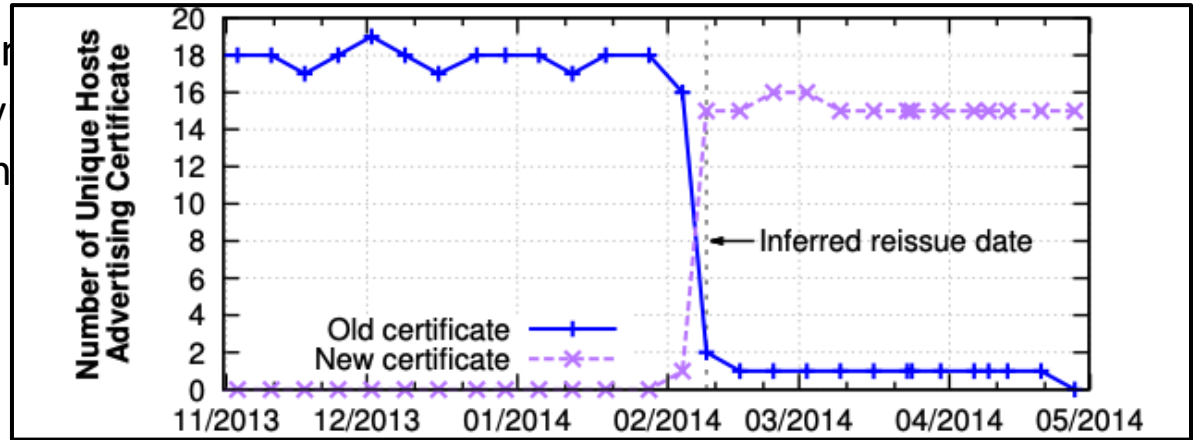


INFERRING HEARTBLEED VULNERABILITY

- Check if a website admin revoked or reissued their certificate
 - It has been running a vulnerable OpenSSL version
 - It has not supported the Max Fragment Length

INFERRING HEARTBLEED VULNERABILITY – CONT'D

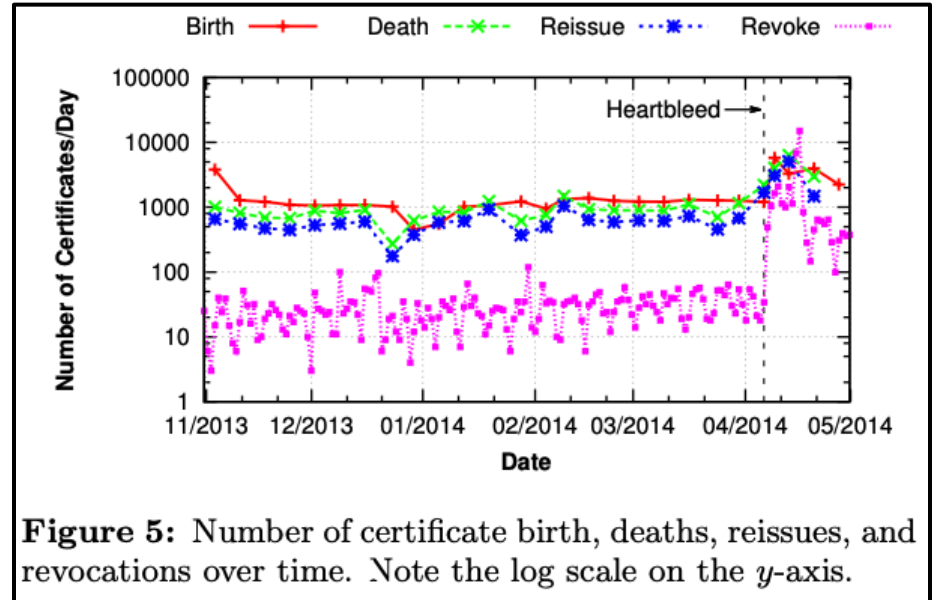
- Check if a website administrator
 - It has been running a vulnerable version
 - It has not supported the vulnerable version



- Vulnerability analysis
 - Certificate birth: the date of the first scan (observing a host advertising the cert)
 - Certificate death: the last date that the number of advertising the cert $> 10\%$ of the max advertising that they observed before
 - Certificate reissue:
 - If the cert dies and they observe a new cert with the same CN within 10 days
 - and If at least one IP address switch from the old cert to the new cert
 - Certificate revocation: when the cert's serial number appears in any CRLs

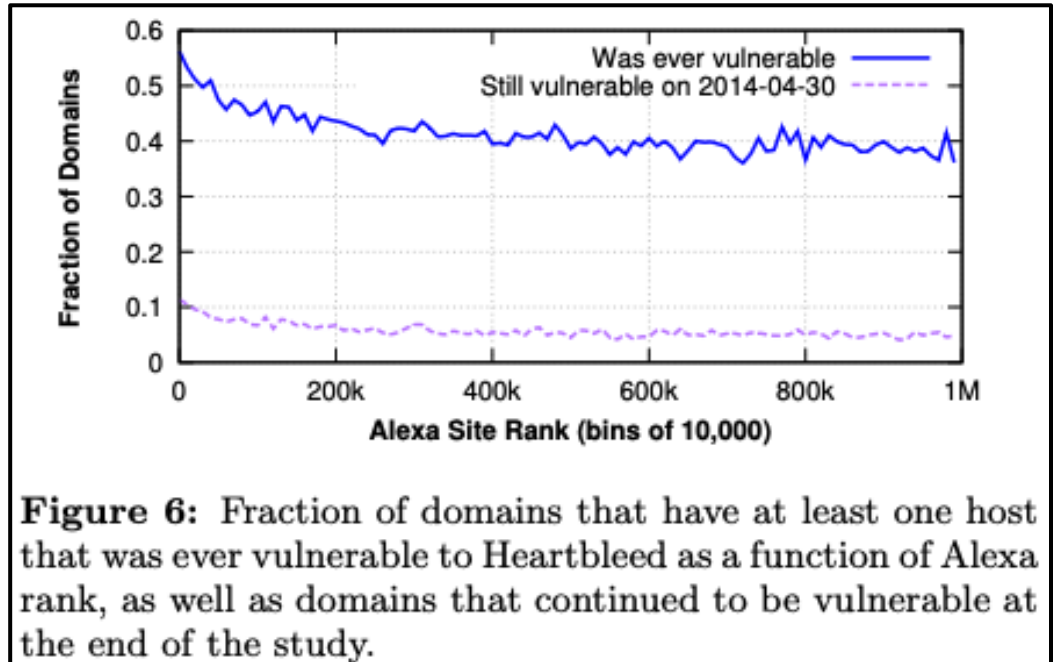
INFERRING HEARTBLEED VULNERABILITY – CONT'D

- Birth, death, reissue, and revocation
 - The number of birth > the number of death
 - A large spike in all four events in the wake of Heartbleed
 - On average, 29 revocations per day before Heartbleed, but this jumps to 1,414 after it



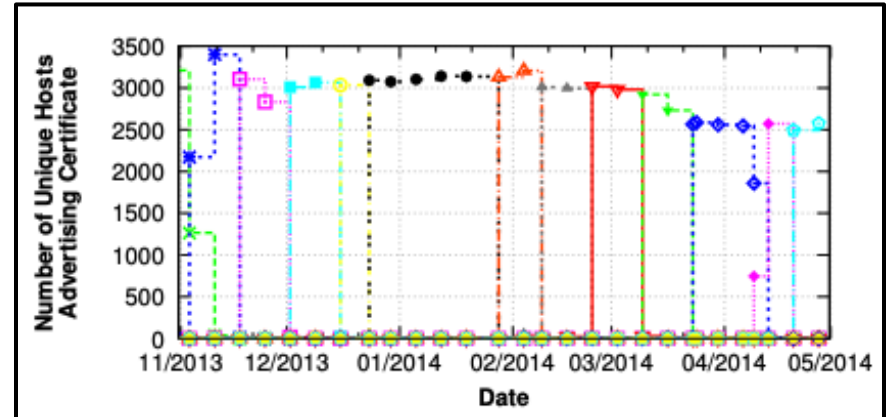
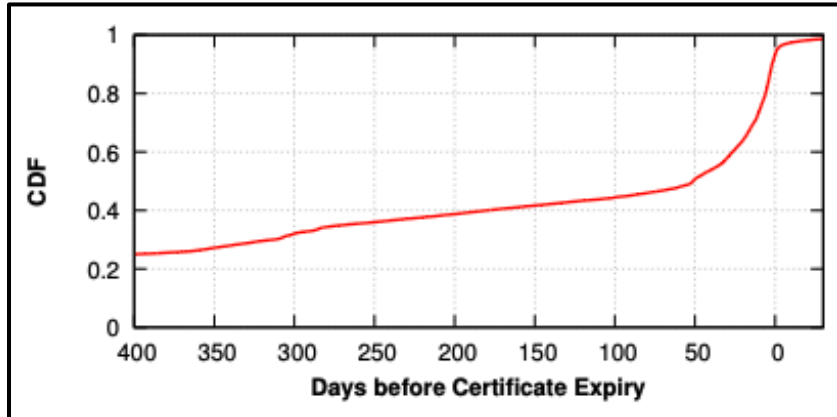
INFERRING HEARTBLEED VULNERABILITY – CONT'D

- Prevalence after the wake
 - The vulnerability has been reduced to less than 10%
 - But that does not mean that our Internet is safe



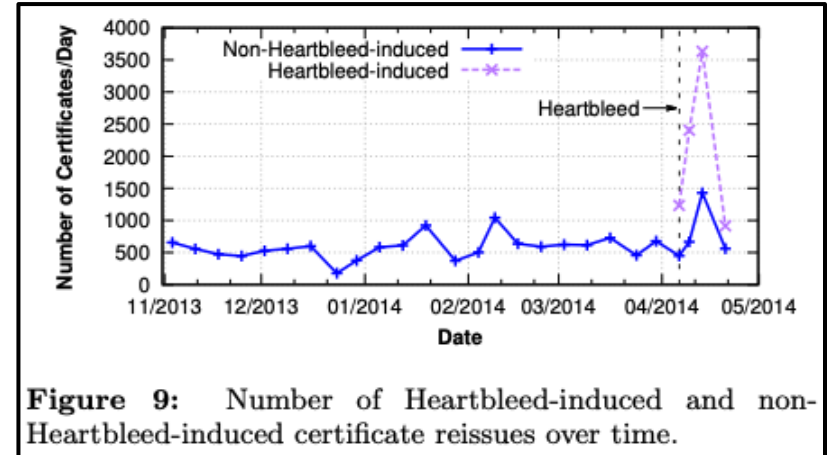
INFERRING HEARTBLEED VULNERABILITY – CONT'D

- Certificate Reissues
 - Generally, 50% of the certificates are re-issued within 60 days
 - A site may periodically reissue certificates as a matter of policy, *e.g.*, google.com



INFERRING HEARTBLEED VULNERABILITY – CONT'D

- Certificate Reissues
 - Generally, 50% of the certificates are re-issued within 60 days
 - A site may periodically reissue certificates as a matter of policy, *e.g.*, google.com
- Heartbleed induced reissues
 - The date of reissue was on and after the Heartbleed
 - > 60 days left until the expiration date
 - No two other issues for the certs with the same common name (CA) before Heartbleed

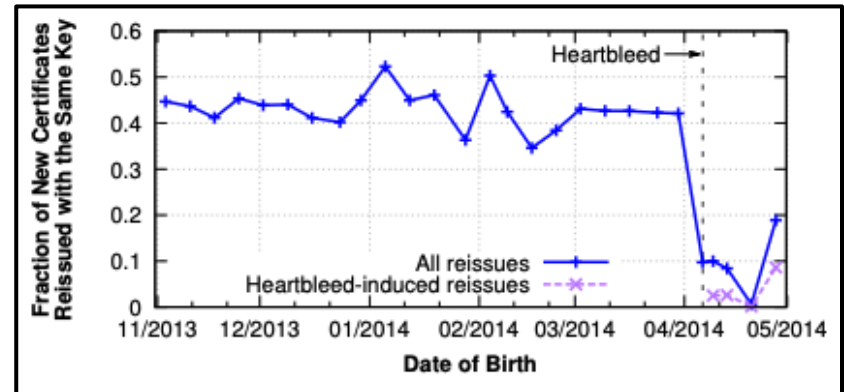


INFERRING HEARTBLEED VULNERABILITY – CONT'D

- Certificate Reissues
 - Generally, 50% of the certificates are re-issued within 60 days
 - A site may periodically reissue certificates as a matter of policy, *e.g.*, google.com
- Heartbleed-vulnerable certificates (that should have been reissued)
 - The date of birth was before Heartbleed
 - The certs won't expire 1 mo after the event
 - The certs were from the vulnerable hosts
 - Results
 - 107,712 vulnerable certificates
 - 26.7% has been re-issued before “1 mo after Heartbleed”
 - 73.3% has not been re-issued

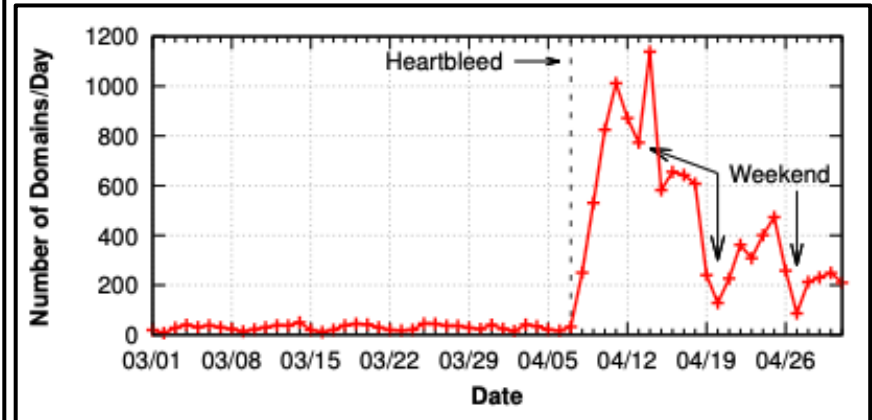
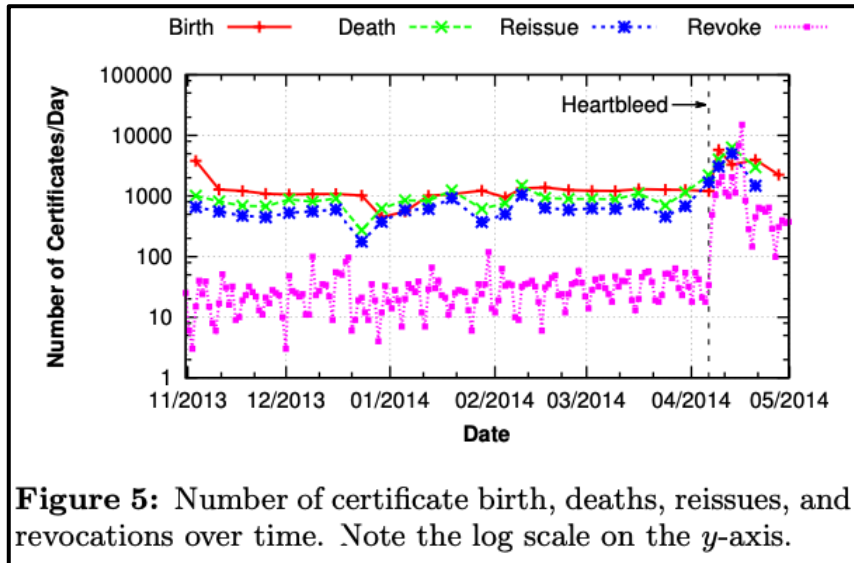
INFERRING HEARTBLEED VULNERABILITY – CONT'D

- Certificate Reissues
 - Generally, 50% of the certificates are re-issued within 60 days
 - A site may periodically reissue certificates as a matter of policy, *e.g.*, google.com
- Heartbleed-vulnerable certificates (that should have been reissued)
 - The date of birth was before Heartbleed
 - The certs won't expire 1 mo after the event
 - The certs were from the vulnerable hosts
 - Results
 - There are certs are re-issued with the *same* key



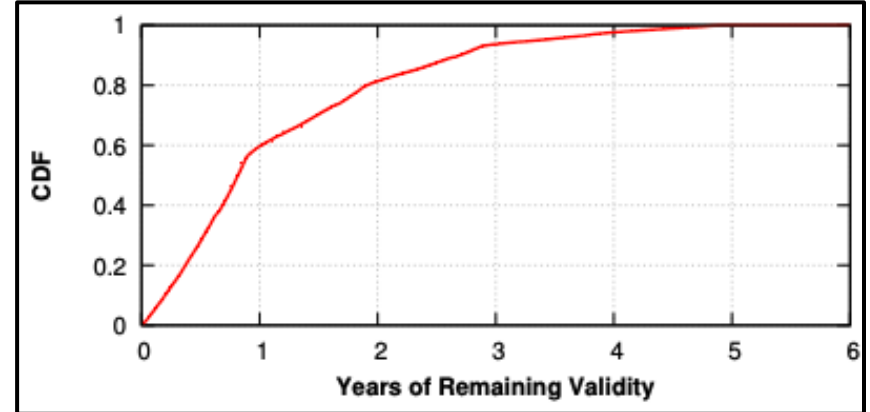
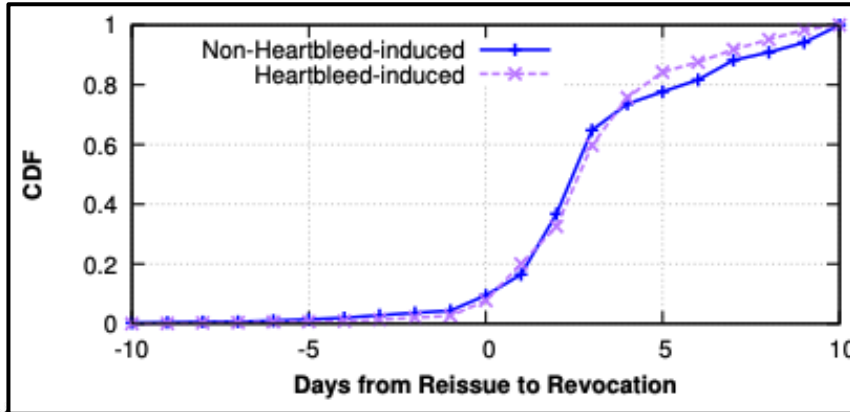
INFERRING HEARTBLEED VULNERABILITY – CONT'D

- Certificate revocation
 - There is a spike in revocation after the wake of Heartbleed
 - Most of the domains that will revoke their certs in direct response to Heartbleed
 - There are “dips” in the revocation rate (particularly during the weekend)



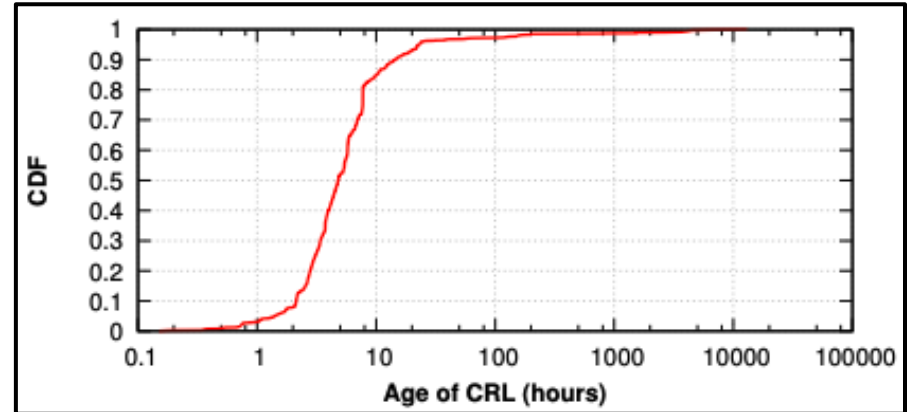
INFERRING HEARTBLEED VULNERABILITY – CONT'D

- Certificate revocation
 - 40% of the retired certificates are revoked (2-3% of the total revocation)
 - Revocation and reissue do not happen at the same time... what? (speed difference)
 - Reissues have been done, but revocations were not done (the long term vuln.)



INFERRING HEARTBLEED VULNERABILITY – CONT'D

- Certificate revocation lists (CRLs)
 - An increase in the number of revocation reasons “key compromised” (0.4 -> 1.18%)
 - 85% of the times, 85% of the revocations are available on clients after 10 hours
 - CAs could revoke certificates as often as every few hours
 - The delay was because of the human-in-the-loop
 - It’s unclear about the client’s impact



DOMAIN NAME SYSTEMS (DNS)

DOMAIN NAME SYSTEM

- A hierarchical and distributed name service that
 - Offers a naming system for computers, services, and other resources on the Internet
 - Associates various information (e.g., IPv4 addr.) to domain names
- “Records”: the associations
 - Each record has a time-to-live (TTL), e.g., in cache
 - It supports different types, e.g.,
 - A/AAAA record: name to IPv4/IPv6 (such as sanghyun-hong.com to 123.456.789.012)
 - CNAME record: name to canonical name (myspace.* to facebook.com/alice)
 - MX record: main exchanger records
 - NS record: nameserver records
 - TXT record: text record (e.g., _github-pages-challenge... to c578365nsjd...)

DOMAIN NAME SYSTEM – CONT'D

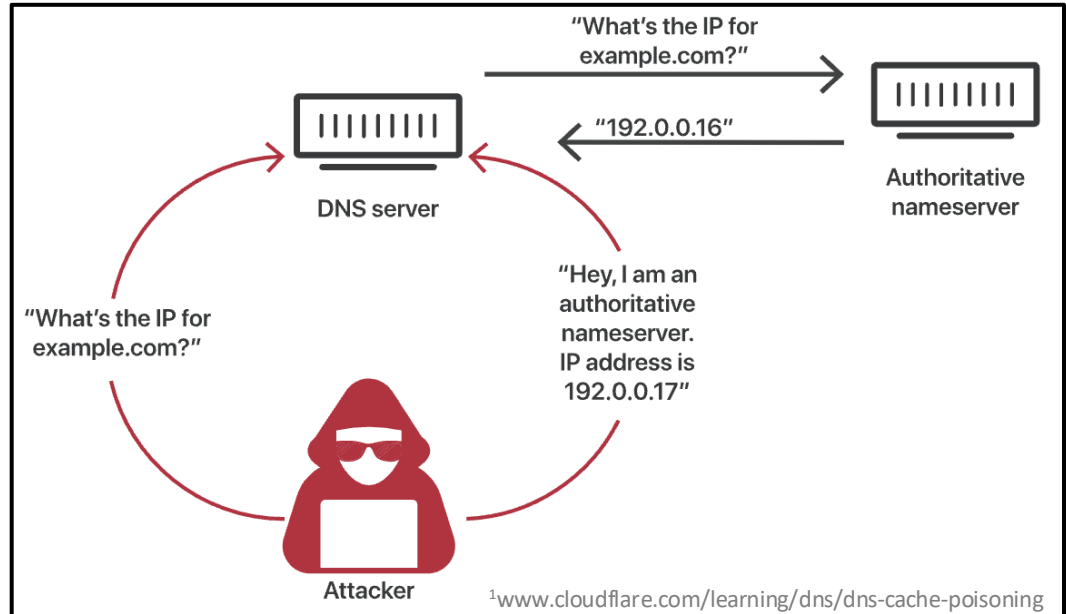
- How does it work?
 - You enter www.sanghyun-hong.com
 - Your browser searches its, OS, or router caches
 - If the value (e.g., IPv4 of the website) is found, then access it -> Done
 - Your browser access to DNS resolver
 - The DNS resolver finds out and accesses
 - The name servers for the TLD (.com)
 - The authoritative name servers for the domain (sanghyun-hong.com)
 - The domain name server for my website (www.sanghyun-hong.com)
 - The IPv4 (or IPv6) address of my website
 - Returns the IP address
 - Your browser accesses the IPv4 and receives my webpages

DOMAIN NAME SYSTEM – CONT'D

- How does it work?
 - DNS packets use UDP by default
 - DNS can use TCP packets as a fallback
 - Port #53

DOMAIN NAME SYSTEM – VULNERABILITIES

- DNS **cache poisoning/spoofing**
 - An adversary may impersonate the DNS nameservers
 - If impersonate the TLD server, ask the IP of sanghyun-hong.com
 - If impersonate the authoritative nameserver, returns a fake IP



DOMAIN NAME SYSTEM – VULNERABILITIES

- DNS **cache poisoning/spoofing**
 - An adversary may impersonate the DNS nameservers
 - If impersonate the TLD server, ask the IP of sanghyun-hong.com
 - If impersonate the authoritative nameserver, returns a fake IP
- Once poisoned
 - User requests to sanghyun-hong.com
 - The DNS server (resolver) will reply with the fake IP
 - The fake IP is highly likely to be associated with malicious website

DOMAIN NAME SYSTEMS SECURITY EXTENSIONS (DNSSEC)

DNSSEC

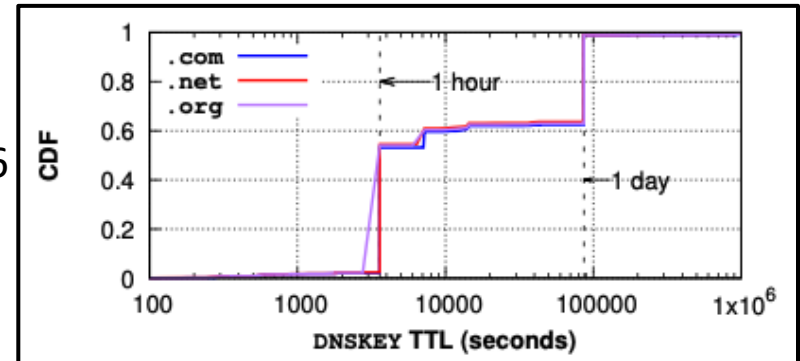
- A security extension that
 - Secures data exchanged in the DNS in networks
 - Adds cryptographic signatures to existing DNS records
 - Stores digital certificates with “records” (e.g., A, AAAA, CNAME, etc.)
- DNSSEC requires a few more DNS records
 - RRSIG record: contains cryptographic signatures
 - DNSKEY record: contains a public key for signing signatures
 - DS record: contains the hash of a DNSKEY record
 - NSEC (NSECC3): explicit denial-of-existence of a DNS record
 - CDNSKEY (CDS): a child zone that requests updates to DS records in the parent

DNSSEC – HOW IT WORKS?

- DNS Zone owner
 - Generates a **private** (key signing key) and **public** key (zone signing key)
 - The **private** key is used to sign all DNS records within the zone
 - Each signed DNS record is accompanied by an RRSIG record (containing the signature)
 - The **public** key(s) are published in the DNS zone
- User / client
 - The DNS resolver retrieves the signed DNS records and their RRSIG records
 - The resolver retrieves the **public** key from the DNS zone
 - The resolver uses the public key to validate the signature on the DNS record
 - The resolver runs this validation through the DNS hierarchy
 - Upon completion of the validation, the resolver will send the records

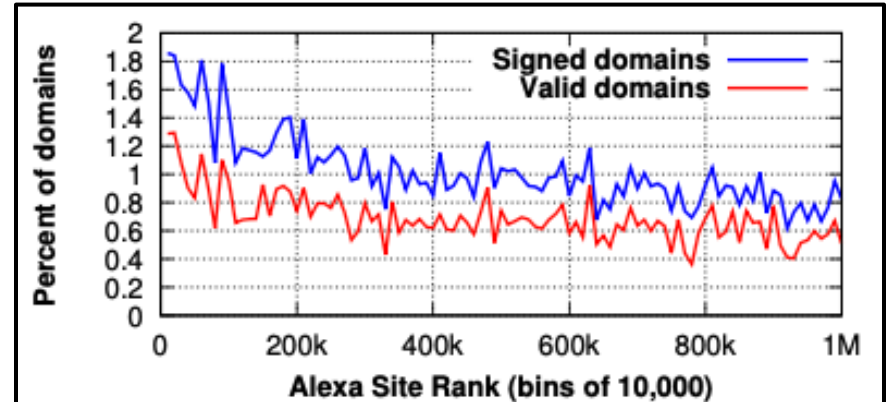
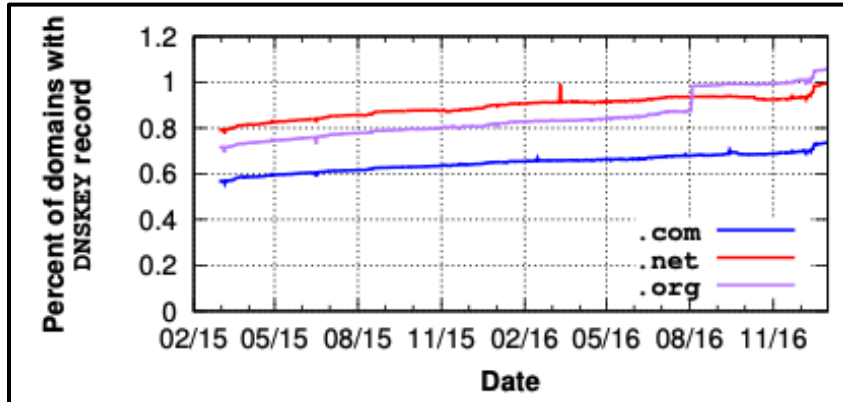
DNSSEC – DEPLOYMENT AND MANAGEMENT

- Research questions
 - How widely is DNSSEC deployed?
 - How often are DNSSEC records correctly published and managed?
 - How are DNSSEC cryptographic keys managed and maintained?
- Dataset
 - .com, .net and .org TLDs (150M domains)
 - 64% of the Alexa Top-1M
 - 75% of the Alexa Top-1K
 - Daily dataset : Mar 1, 2015 – Dec 31, 2016
 - Hourly dataset: Sep 29, 2016 – Dec 31, 2016



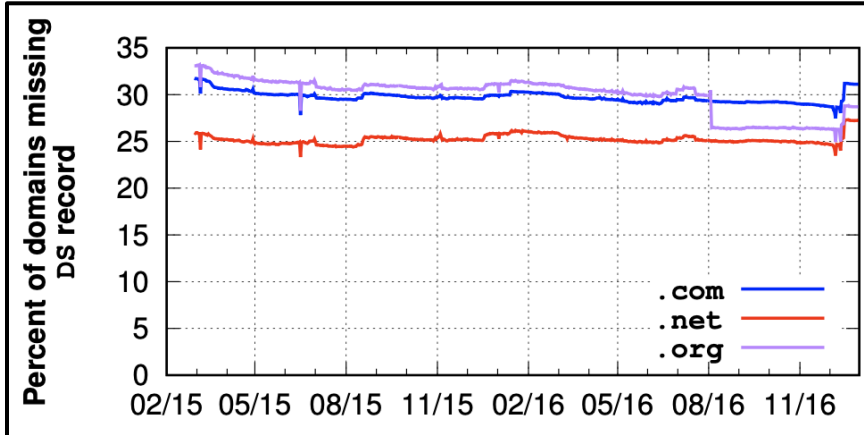
DNSSEC – DEPLOYMENT AND MANAGEMENT

- Prevalence
 - DNSSEC deployment is rare (0.6 – 1.0% of domains, .com and .org respectively)
 - The deployment increases over time (0.75 to 1.0%)
 - There are spikes due to actions by a few authoritative name servers (.org)
 - But this means that a few authoritative name servers are responsible for the dep.
 - Popular websites are more likely to sign their domains



DNSSEC – DEPLOYMENT AND MANAGEMENT

- Missing records
 - 28 – 32% domains do not have a DS record
 - 15 authoritative name servers cover 83% of domains collected
 - 4 authoritative name servers fail to publish DS records for nearly all of their domains
 - Drop in .org is due to hyp.net publishing 11k signed domains, and spike was caused by Domain Monster, 37k new domains



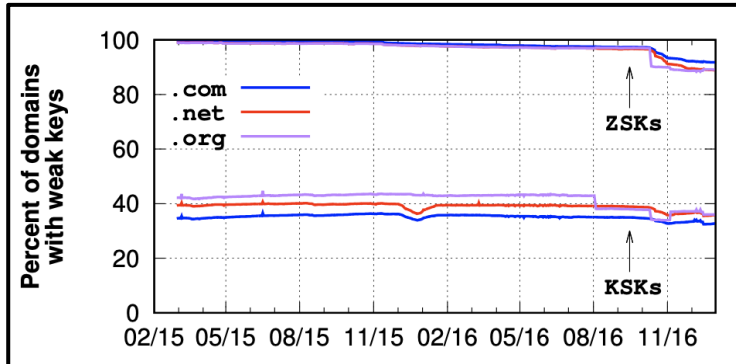
Name servers	Number of domains Signed	w/ DS	DS Publishing Ratio
*.ovh.net	316,960	315,204	99.45%
*.loopia.se	131,726	1	0.00%
*.hyp.net	94,084	93,946	99.85%
*.transip.net	91,103	91,009	99.90%
*.domainmonster.com	60,425	4	0.01%
*.anycast.me	52,381	51,403	98.13%
*.transip.nl	47,007	46,971	99.92%
*.binero.se	44,650	17,099	38.30%
*.ns.cloudflare.com	28,938	17,483	60.42%
*.is.nl	15,738	11	0.07%
*.pccextreme.nl	14,967	14,801	98.89%
*.webhostingserver.nl	14,806	10,655	71.96%
*.registrar-servers.com	13,115	11,463	87.40%
*.nl	12,738	12,674	99.50%
*.citynetwork.se	11,660	13	0.11%

DNSSEC – DEPLOYMENT AND MANAGEMENT

- Incorrect records
 - 99.5% of domains, where RRISG validation for SOA records fails, are valid
 - 99.9% of DS records are valid

DNSSEC – DEPLOYMENT AND MANAGEMENT

- Key management
 - Two domains share a privacy key?
 - 99.95% of keys are used for one domain
 - 0.04% private keys and 0.05% public keys are shared by more than one domain
 - One private and public keys are shared over 132k domains
 - Weak keys?
 - < 1024-bit RSA keys are weak
 - < 2048-bit DSA keys are weak



Name servers	KSK		ZSK	
	Domains	Keys	Domains	Keys
*.others	151,733	157,533	152,144	188,482
*.ovh.net	316,888	318,036	316,887	326,011
*.loopia.se	133,258	199	133,258	217
*.hyp.net	94,888	119,150	94,885	119,161
*.transip.net	93,819	93,774	93,818	187,129
*.domainmonster.com	60,984	60,991	60,984	121,939
*.anycast.me	55,936	56,075	55,936	58,296
*.transip.nl	45,676	45,648	45,675	91,161
*.binero.se	44,963	49	44,963	54
*.ns.cloudflare.com	28,469	239	28,469	214
*.nl	12,837	12,834	12,836	25,512
*.pcextreme.nl	15,210	15,192	15,210	28,654
*.webhostingserver.nl	15,023	15,019	15,023	22,741
*.registrar-servers.com	13,183	13,043	13,181	12,998
*.is.nl	11,945	11,978	11,945	23,790
*.citynetwork.se	11,702	21	11,702	28

Thank You!

Sanghyun Hong

<https://secure-ai.systems/courses/Sec-Grad/current>



Oregon State
University



TRUE AI
Trustworthy and Responsible AI